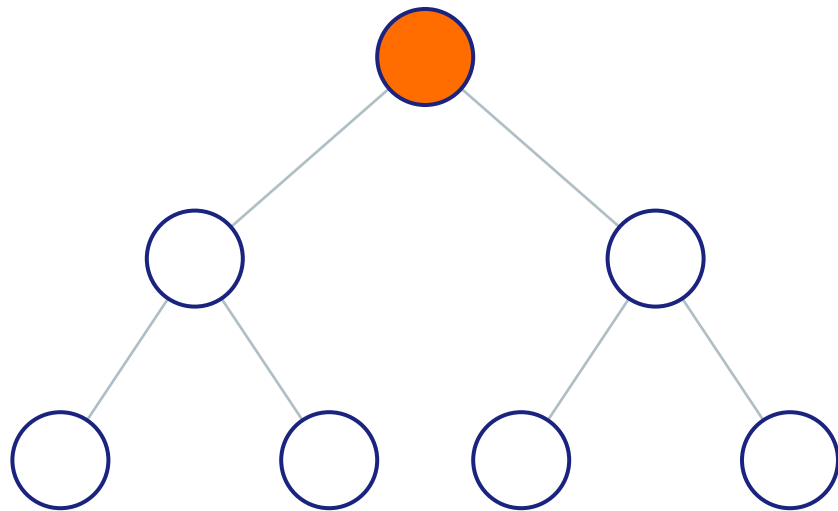


ESTRUTURAS DE DADOS

Árvores

Conceitos fundamentais, árvores binárias, percursos, ABB e análise de complexidade.



Conceitos Fundamentais de Árvores

As árvores são estruturas de dados não lineares e hierárquicas, amplamente utilizadas na ciência da computação para representar relações hierárquicas entre elementos. Diferentemente de estruturas lineares como listas e filas, as árvores permitem uma organização mais complexa e eficiente para certas operações [1].

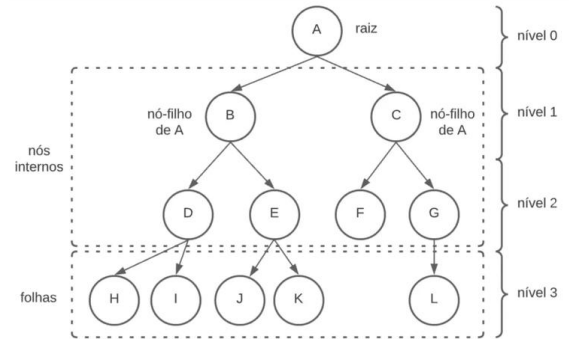
[1] **Nó:** Cada elemento em uma árvore é chamado de nó. Um nó pode conter dados e referências a outros nós.

[2] **Raiz:** É o nó superior da árvore, o único que não possui um nó pai. A partir da raiz, todos os outros nós são acessíveis são acessíveis [1].

[3] **Folha:** Um nó folha é aquele que não possui filhos; são os nós terminais da árvore [1].

[4] **Altura:** A altura de um nó é o número de arestas no caminho mais longo entre esse nó e uma folha descendente. A altura da árvore é a altura da raiz [2].

[5] **Profundidade:** A profundidade de um nó é o número de arestas no caminho entre a raiz e esse nó. A raiz tem profundidade 0 [2].



Árvores Binárias

Uma árvore binária é um tipo especial de árvore onde cada nó possui no máximo dois filhos, geralmente referidos como filho esquerdo e filho direito. Essa restrição simplifica muitas operações e é a base para estruturas mais avançadas [1].

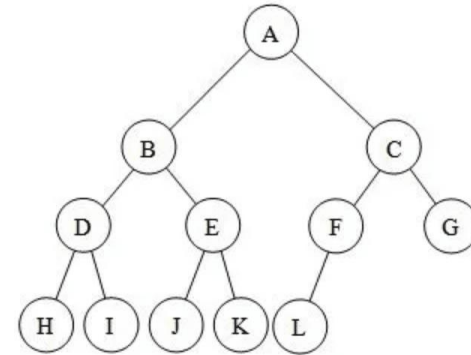
Tipos de Árvores Binárias:

[1] **Árvore Binária Cheia:** Cada nó, exceto as folhas, possui exatamente dois filhos [2].

[2] **Árvore Binária Completa:** Níveis preenchidos, exceto possivelmente o último (da esquerda para a direita) [2].

[3] **Árvore Binária Perfeita:** Cheia e com todas as folhas no mesmo nível [2].

[4] **Árvore Binária Degenerada:** Cada nó pai tem apenas um filho, similar a uma lista encadeada [1].



Representação na Memória:

Representadas por **ponteiros** (nós como objetos dinâmicos) ou **arrays** (eficiente para árvores completas, onde os filhos de i estão em $2i+1$ e $2i+2$) [3].

Percursos em Árvores Binárias

Métodos para visitar todos os nós de uma árvore binária de forma sistemática [1]:

Pré-ordem (Pre-order)

Visita a raiz, depois a subárvore esquerda e a direita. Útil para copiar a árvore [1].

- [1] Visita a raiz.
- [2] Percorre a subárvore esquerda.
- [3] Percorre a subárvore direita.

Em-ordem (In-order)

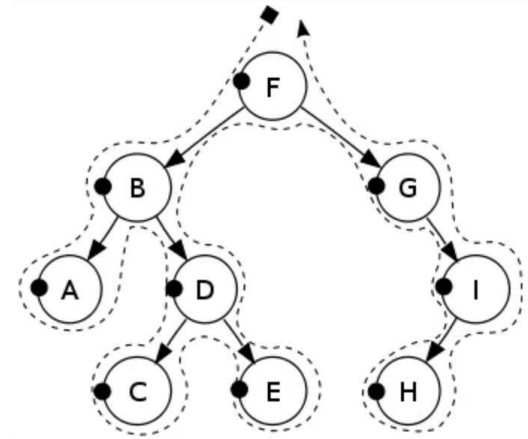
Esquerda, Raiz, Direita. Em ABBs, retorna os elementos em ordem crescente [1].

- [1] Percorre a subárvore esquerda.
- [2] Visita a raiz.
- [3] Percorre a subárvore direita.

Pós-ordem (Post-order)

Esquerda, Direita, Raiz. Útil para deletar a árvore [1].

- [1] Percorre a subárvore esquerda.
- [2] Percorre a subárvore direita.
- [3] Visita a raiz.



ÁRVORE BINÁRIA DE BUSCA (ABB)

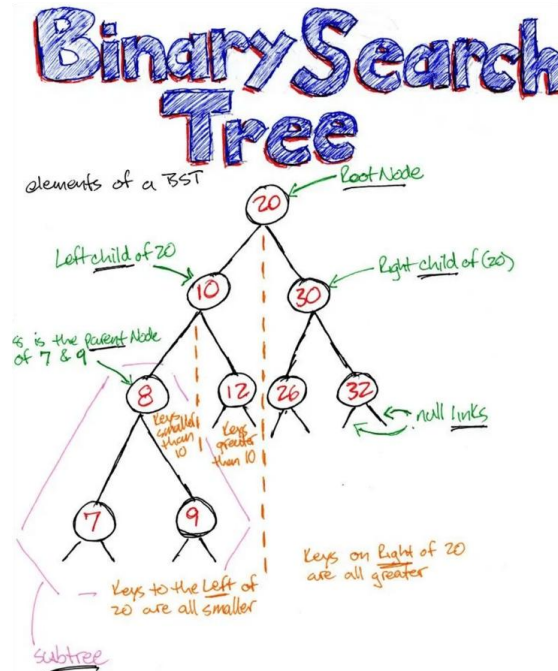
Definição

A ABB é uma estrutura de dados organizada para permitir buscas, inserções e remoções eficientes. É baseada na ideia de busca binária em uma estrutura dinâmica [1].

Propriedades

- [1] Subárvore Esquerda: Contém apenas nós com chaves menores que a raiz.
- [2] Subárvore Direita: Contém apenas nós com chaves maiores que a raiz.
- [3] Recursividade: Cada subárvore também deve ser uma ABB [1].

Visualização Técnica



Vantagens

- [1] Eficiência: Complexidade média $O(\log n)$ para operações principais [2].
- [2] Ordenação: O percurso em ordem resulta em dados ordenados.

Aplicações

- [1] Bancos de Dados: Indexação de registros para busca rápida.
- [2] Sistemas de Arquivos: Organização hierárquica de diretórios.
- [3] Compiladores: Construção de tabelas de símbolos [2].

Operações em Árvores Binárias de Busca

Busca e Inserção

[1] **Busca (Search)**: Inicia na raiz. Compara o valor: se menor, vai para a esquerda; se maior, para a direita. Repete até encontrar ou atingir um nó nulo.

[2] **Inserção (Insertion)**: Semelhante à busca. Percorre a árvore até encontrar uma posição vazia (folha) que respeite a propriedade da ABB.

Remoção (Deletion)

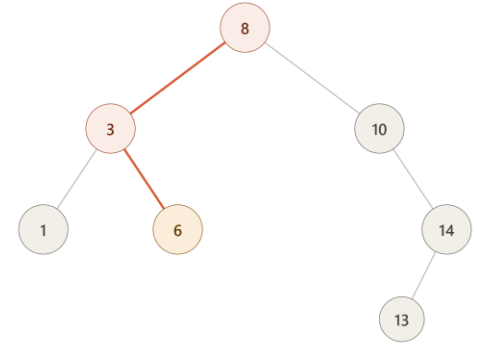
A remoção possui três cenários fundamentais:

[1] **Nó Folha**: Simplesmente removido da árvore.

[2] **Nó com um Filho**: O filho assume a posição do pai removido.

[3] **Nó com dois Filhos**: Substituído pelo sucessor em-ordem em-ordem (menor da subárvore direita) ou predecessor em-ordem (maior da esquerda).

Visualização Técnica



Busca por 6: 8 → 3 → 6 (encontrado)

Nó folha



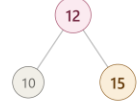
Folha: removida direto

Remoção: os três casos
Um filho



Filho assume o lugar

Dois filhos



Sucessor o substitui

Código e Complexidade

Implementação em Java

```
pseudocodigo_insercao_abb

1  ESTRUTURA Nó
2      chave: inteiro
3      esquerda, direita: Nó
4
5  FUNÇÃO inserir(raiz, chave)
6      SE raiz = nulo ENTÃO
7          raiz ← novo Nó(chave)
8          RETORNE raiz
9
10     SE chave < raiz.chave ENTÃO
11         raiz.esquerda ← inserir(raiz.esquerda, chave)
12     SENÃO SE chave > raiz.chave ENTÃO
13         raiz.direita ← inserir(raiz.direita, chave)
14
15     RETORNE raiz
```

Análise Big-O

Operação	Médio	Pior
Busca	$O(\log n)$	$O(n)$
Inserção	$O(\log n)$	$O(n)$
Remoção	$O(\log n)$	$O(n)$

[1] $O(\log n)$: Ocorre em árvores balanceadas. A altura é proporcional ao logaritmo do número de nós.

[2] $O(n)$: Ocorre em árvores degeneradas (similares a listas), onde a altura é igual ao número de nós.

